Exact algebraic computation

From `Root` Objects to Radicals

A systematic construction, certified branches, and a practical Wolfram Language/Python package

Technical report prepared for Vladimir Reshetnikov

8 September 2026 • Version 0.1.0

Abstract

An exact algebraic number is expressible by radicals precisely when the Galois group of its minimal polynomial is solvable. Turning this characterization into useful software requires considerably more than testing a group: one must construct the intermediate fields, produce nonzero Kummer generators, preserve the intended complex embedding, and distinguish a resource failure from a mathematical obstruction.

This report develops and implements a deterministic splitting-field construction. A finite trace-and-Fourier search supplies a generator at each cyclic prime-degree step; exact rational linear algebra expresses the resulting radicands in the preceding field. Root separation certifies all choices of principal radical branches. A replayable certificate separates the algebraic result from the search that found it.

The general constructor is preceded by exact low-degree, Dickson-polynomial, and pair-sum-resolvent methods. These recover both examples in the original question and substantially improve the practical range. The distribution includes a tested SymPy implementation, a Wolfram Language interface with native shortcuts and exact final verification, regression tests, and the supplied experimental notebook. Mathematical completeness is stated separately from practical bounds and from the actual scope of validation.

Validation boundary. Python execution and exact certificate replay were performed. The Wolfram Language source and prospective kernel tests are included, but could not be executed in a Wolfram kernel in the build environment. The generic method is a transparent reference implementation, not a claim of competitive performance with specialized Galois-theory systems.

Contents

1 The problem and the delivered contract

The motivating question asks why ToRadicals leaves some solvable algebraic numbers as Root objects, and whether a systematic workaround is possible [1]. Its two initial examples are

$$\alpha = \texttt{Root[-1 - ##\^{}2 - ##\^{}3 + ##\^{}4 + ##\^{}6 \&, 2]}, \tag{1.1}$$

$$\beta = \texttt{Root[6 + 25 ## - 25 ##\^{}3 + 5 ##\^{}5 \&, 5]}. \tag{1.2}$$

The accompanying notebook explores auxiliary algebraic fields over which their defining polynomials acquire factors of smaller degree. This is a valuable computational idea, but a fixed collection of guessed extensions is not a completeness argument.

Wolfram's documented ToRadicals conversion covers algebraic roots of degree at most four and explicitly notes that some higher-degree roots admitting radicals are not converted [2]. The relevant distinction is therefore not "degree greater than four means impossible." Degree is neither a sufficient obstruction nor, by itself, a useful measure of the size of a radical expression.

1.1 Input and output

The mathematical input is a specified algebraic number $a \in \overline{\mathbb{Q}} \subset \mathbb{C}$, with its embedding fixed. The Python interface accepts a rational polynomial and either all its distinct roots or a selected root in SymPy's ordering. The Wolfram interface accepts an exact parameter-free algebraic expression, normalizes it with RootReduce, and obtains its minimal polynomial over $\mathbb{Q}$. In particular, the intended inputs include polynomial Root objects. Parametric root families, approximate numbers, and transcendental roots are outside this release's contract.

An output radical expression belongs to the recursively generated grammar

$$e ::= q \mid i \mid e_1 + \cdots + e_k \mid e_1 \cdots e_k \mid e^r, \qquad q, r \in \mathbb{Q}.$$

Every rational power has its ordinary principal complex meaning. Negative integral powers provide division when the denominator is nonzero. Neither Root, AlgebraicNumber, trigonometric functions, nor an unevaluated root-selection operator is accepted as a radical output. Roots of unity are written as

$$\zeta_m = (-1)^{2/m} = e^{2\pi i/m}, \qquad m > 1, \qquad \zeta_1 = 1. \tag{1}$$

The exponential in this equality explains the value; it is not the emitted expression.

A successful result certifies the specified conjugate, not just some root of the same polynomial. For the Wolfram interface the final acceptance condition is literally

```
RadicalExpressionQ[e] && RootReduce[e - a] === 0
```

with exact arithmetic and error handling. A numerical residual is not an alternative acceptance condition.

1.2 Five outcomes that must not be conflated

Status	Meaning
`Success`	A radical expression was constructed and its conjugate was certified.
`NotSolvable`	An exact Galois-group computation established nonsolvability for the relevant irreducible polynomial.
`SearchExhausted`	The requested finite set of shortcuts did not succeed. This is not a nonsolvability result.
`ResourceLimit`	A time or configured field-degree limit was reached. Solvability remains undetermined.
`VerificationFailure`	An expected exact identity or branch match failed. The candidate is not returned as a verified answer.

Invalid input, missing dependencies, and unexpected backend failures have their own diagnostic statuses. In particular, an exception from a Galois-group routine is never translated into `NotSolvable`.

2 What the experimental notebook contributes

The supplied archive contains `FindExtension.nb`. The distribution preserves it unchanged under `original/`. A non-evaluating parser also extracts 220 input cells into a display-oriented transcription, retaining source-line markers. The extraction script parses notebook brackets and strings; it does not execute the notebook. Unsupported display boxes are marked rather than silently turned into program text.

Several ideas in the notebook are sound starting points: finding a quadratic or cubic factor over an auxiliary extension, projecting conjugates to their real or imaginary parts, and searching for low-degree symmetric combinations of roots. The main changes in the present package concern the way those ideas are made deterministic and checked.

Notebook component	Observed limitation	Replacement in this release
`findExtension`, `findExtension3`	Elimination searches are useful but do not identify a complete class of extension generators. Selection depends on expression complexity.	An exact pair-sum resultant supplies a finite, reproducible family; a Galois–Kummer construction handles the unrestricted mathematical problem.
`factor`, `solve`	Target selection uses `PossibleZeroQ`; factor handling partly depends on whether the expression head is `Times`.	Exact factor lists, quotient-field identities, and exact or separation-certified conjugate matching.
`fuzz`	Random subsets and fixed 400-digit reconstruction are exploratory. Its unbounded loop has no exhaustion condition after the finite subset collection is consumed.	No random or numerical reconstruction is needed by the general algorithm. Limits and exhaustion are explicit.
`radicalDepth`	The zero-depth rule includes expressions with no rational powers, so a bare `Root` can receive depth zero.	A strict output grammar first distinguishes a radical expression from an opaque algebraic representation. Minimal radical depth is not claimed.

Notebook component	Observed limitation	Replacement in this release
Complexity ranking	Private implementation symbols are used. `Simplify‘SimpifyCount` appears with a spelling different from `Simplify‘SimplifyCount`.	Public algebraic primitives and deterministic ordering. No private complexity function is required.
Interactive simplification	Global definitions, history references, and a hard-coded exceptional degree-nine root complicate reproducibility.	Modules with explicit inputs, a portable JSON protocol, and independently runnable tests.

These observations refer to the supplied notebook, particularly the input definitions beginning at notebook source lines 21, 65, 126, 252, 315, 375, 436, and 560. They are not a claim that every exploratory cell is defective. Exploratory notebooks appropriately permit guesses; a conversion package must additionally state what a guess proves and when a search has finished.

The notebook contains substantially larger examples than the regression suite in this release. They were inspected as source material, not all executed or all solved. No claim of full notebook-corpus coverage is made.

3 The exact mathematical criterion

3.1 One selected root, not an arbitrary defining polynomial

Let $f = \mathrm{minpol}_{\mathbb{Q}}(a)$ be the monic irreducible minimal polynomial of the specified number a, and let L be its splitting field in $\mathbb{C}$.

Theorem 3.1 (Radical criterion)**.** *The number a is expressible by radicals over $\mathbb{Q}$ if and only if $\mathrm{Gal}(L/\mathbb{Q})$ is solvable.*

Proof. If the Galois group is solvable, adjoining suitable roots of unity and following a cyclic prime composition series gives a radical tower containing L. Sections 7–9 give the constructive direction explicitly.

Conversely, suppose a belongs to a finite radical tower. Adjoin the finitely many required roots of unity, and form normal closures stage by stage. At a radical step $u^n = b$, the conjugates are obtained by multiplying appropriate roots by roots of unity; over a field containing those roots of unity the new Galois kernels are abelian. Induction shows that the normal closure lies in a finite solvable Galois extension of $\mathbb{Q}$. Every conjugate of a lies in that extension, hence so does L. Its Galois group is a quotient of a solvable group and is solvable. □

The use of the *minimal* polynomial matters. For example, the polynomial

$$(X - 2)(X^5 - X - 1)$$

has a rational root although its quintic factor has nonsolvable Galois group. A request for the root 2 must not be rejected because another factor is nonsolvable. The Python `solve_root` function selects the irreducible factor before invoking the general constructor. `solve_polynomial`, by contrast, requests every distinct root and consequently fails to produce a complete radical list when any factor is nonsolvable.

Repeated factors do not change radical solvability. The all-roots interface replaces the input by its monic square-free part. The selected-root interface counts input multiplicities when interpreting its index and then works with the selected irreducible factor.

3.2 Why working only inside $\mathbb{Q}(a)$ is insufficient

The extension $\mathbb{Q}(a)/\mathbb{Q}$ need not be normal, even for a very simple radical. For $a = \sqrt[3]{2}$ it is a real cubic field, while the splitting field also contains ζ_3. Trying to construct a chain of Galois subextensions entirely inside $\mathbb{Q}(a)$ can therefore fail for a number whose radical representation is immediate.

The general algorithm deliberately works in a normal extension. This is mathematically convenient and computationally expensive. The practical shortcuts often avoid that expense, but they do not replace the normal-extension argument that establishes completeness.

4 A practical front end: exact identities before large fields

The default solver tries inexpensive structural methods before constructing a splitting field. Each shortcut has an exact algebraic witness. Its failure merely passes control to the next method.

4.1 Degree at most four

Linear, quadratic, cubic, and quartic formulas are used when they provide expressions in the accepted grammar. The Python implementation uses SymPy's exact polynomial formulas; the native Wolfram path first tries `ToRadicals`. Returned candidates still undergo conjugate selection.

No real-radical-only requirement is imposed. In particular, real roots of irreducible cubics with three real roots can require complex intermediate quantities in their usual radical formulas. Suppressing those quantities with a real-power convention would change the problem and can select the wrong branch.

4.2 Dickson-polynomial recognition

Define

$$D_0(X, a) = 2, \qquad D_1(X, a) = X, \qquad D_n(X, a) = XD_{n-1}(X, a) - aD_{n-2}(X, a). \tag{2}$$

The identity that makes these polynomials useful is

$$D_n\left(u + \frac{a}{u}, a\right) = u^n + \left(\frac{a}{u}\right)^n, \qquad u \neq 0. \tag{3}$$

It follows immediately by checking the first two cases and using the recurrence.

Suppose the monic input polynomial can be recognized exactly as

$$f(X) = D_n(X + h, a) + c, \qquad h, a, c \in \mathbb{Q}. \tag{4}$$

Put

$$t = \frac{-c + \sqrt{c^2 - 4a^n}}{2}, \qquad u = t^{1/n}. \tag{5}$$

If this choice gives $t = 0$, use the other quadratic root. The degenerate case in which both choices vanish is handled by factorization rather than division by zero. For nonzero u, all roots are among

$$x_k = \zeta_n^k u + \frac{a}{\zeta_n^k u} - h, \qquad 0 \leq k < n. \tag{6}$$

Indeed, $t^2 + ct + a^n = 0$, so substituting (6) in (3) gives $D_n(x_k + h, a) = -c$. In the square-free case the resulting set supplies all roots. The coupled reciprocal a/u is important: independently selecting principal nth roots for both terms can violate the required product relation.

Recognition is deterministic. If the coefficient of X^{n-1} in the monic polynomial is b_{n-1}, set $h = b_{n-1}/n$ and depress the polynomial by substituting $X - h$. Its coefficient of X^{n-2} is $-na$. The remaining constant determines c; all coefficients are then compared exactly with the recurrence. Binomials are included through $a = 0$. This shortcut has no degree-six restriction.

4.3 The original quintic

For the second example,

$$\frac{1}{5}f(X) = X^5 - 5X^3 + 5X + \frac{6}{5} = D_5(X, 1) + \frac{6}{5}.$$

Consequently, with principal powers,

$$u = \left(\frac{-3 + 4i}{5}\right)^{1/5}, \qquad \boxed{\beta = u + u^{-1}}. \tag{7}$$

The quantity inside the fifth root lies on the unit circle in the upper half-plane. Thus u^{-1} is also the principal fifth root of its complex conjugate, and (7) equals

$$\left(\frac{-3 + 4i}{5}\right)^{1/5} + \left(\frac{-3 - 4i}{5}\right)^{1/5}.$$

The exact root-selection test identifies it as the fifth, largest real root in the original Wolfram example. Its approximate value is

$$\beta = 1.8070599950854245\ldots.$$

The approximation illustrates the result; neither recognition nor certification depends on that decimal value.

5 Pair-sum resolvents and extension discovery

The notebook's search for a quadratic factor can be made finite by explicitly computing the polynomial of all possible pair traces.

5.1 An exact resultant formula

Let $f(X) = \prod_{i=1}^{n}(X - \alpha_i)$ be monic and square-free. Define

$$R_2(Y) = \prod_{1 \le i < j \le n} (Y - \alpha_i - \alpha_j). \tag{8}$$

This polynomial belongs to $\mathbb{Q}[Y]$ by symmetry. Its degree is $\binom{n}{2}$, with repeated pair sums retained as repeated factors.

Proposition 5.1. *The exact identity*

$$\operatorname{Res}_X\big(f(X), f(Y - X)\big) = 2^n f(Y/2)\, R_2(Y)^2 \tag{9}$$

holds in $\mathbb{Q}[Y]$.

Proof. Since f is monic, the resultant is

$$\prod_i f(Y-\alpha_i) = \prod_{i,j}(Y-\alpha_i-\alpha_j).$$

The diagonal factors give $\prod_i(Y-2\alpha_i) = 2^n f(Y/2)$. Each unordered pair off the diagonal occurs twice, giving $R_2(Y)^2$. □

The implementation divides the resultant by the diagonal polynomial exactly, verifies that the remainder is zero, factors the monic quotient over $\mathbb{Q}$, checks that every factor multiplicity is even, and halves those multiplicities. It never takes a numerical square root of polynomial coefficients.

Pair sums can collide. For roots $-2, -1, 1, 2$, the sum zero occurs twice, and

$$R_2(Y) = Y^2(Y^2-1)(Y^2-9).$$

Discarding multiplicities before removing the square would lose information and could corrupt the construction. This collision is explicitly included in the regression tests.

5.2 From a trace to an actual factor

A root s of a low-degree factor of R_2 is a candidate pair trace. The solver constructs the embedded field $\mathbb{Q}(s)$ and factors f over it. A pair sum does *not* automatically determine the corresponding pair product. Therefore a quadratic factor is accepted only when exact factorization actually produces it. There can be ambiguity when several pairs share a trace; field factorization resolves that ambiguity or the shortcut simply does not succeed.

For a quadratic factor

$$A(s)X^2 + B(s)X + C(s),$$

the coefficients are elements of the particular embedded field $\mathbb{Q}(s)$. After a radical representation of that same s has been certified, their polynomial coordinate expressions are evaluated at its radical form and the quadratic formula is applied. This avoids mixing coefficients from one conjugate field embedding with the radical expression of another.

The Python path solves auxiliary polynomials of degree at most four directly and also recognizes higher-degree Dickson auxiliary polynomials up to the configured extension-degree bound. The native Wolfram shortcut currently limits these auxiliary fields to degree at most four. Neither bounded search is complete by itself.

5.3 A compact solution of the original sextic

For

$$F(X) = X^6 + X^4 - X^3 - X^2 - 1,$$

the pair-sum resolvent factors as

$$\begin{aligned} R_2(Y) = (Y^3 + 4Y - 1)(&Y^{12} - Y^9 + 8Y^8 - 2Y^7 - Y^6 \\ &+ 4Y^5 + 18Y^4 + 9Y^3 + 2Y^2 - 1). \end{aligned} \tag{10}$$

The cubic factor is explained by an especially simple exact identity:

$$\boxed{F(X) = X^3 g(X - X^{-1}), \qquad g(S) = S^3 + 4S - 1.} \tag{11}$$

For each root s of g, the two corresponding roots of F satisfy

$$X^2 - sX - 1 = 0. \tag{12}$$

Thus the pair trace is s and the pair product is -1.

Let

$$v = \left(\frac{1}{2} + \frac{\sqrt{849}}{18}\right)^{1/3}, \qquad s = v - \frac{4}{3v}. \tag{13}$$

Here v is positive real. Direct calculation gives $s^3 + 4s - 1 = 0$. The derivative $g'(S) = 3S^2 + 4$ is positive on the real line, so this is the unique real trace. Equation (12) gives exactly two real roots, one negative and one positive. Therefore the requested second real root is

$$\boxed{\alpha = \frac{s + \sqrt{s^2+4}}{2}, \qquad s = \left(\frac{1}{2} + \frac{\sqrt{849}}{18}\right)^{1/3} - \frac{4}{3\left(\frac{1}{2} + \frac{\sqrt{849}}{18}\right)^{1/3}}.} \tag{14}$$

Numerically, $\alpha = 1.13068544546204059\ldots$. The other four roots are obtained by using the other two cubic traces and both quadratic signs. The package verifies all six conjugates.

5.4 A degree-ten example

The polynomial

$$\begin{aligned} P(X) = X^{10} - 15X^8 + 90X^6 - 4X^5 - 270X^4 - 120X^3 \\ + 405X^2 - 180X - 239 \end{aligned}$$

is obtained by eliminating Y from $Y^5 = 2$ and $(X - Y)^2 = 3$. Its pair-sum resolvent has the factor $S^5 - 64$. A root $s = 2\sqrt[5]{2}$ yields the quadratic factor

$$X^2 - sX + \frac{s^2}{4} - 3,$$

and hence the roots $s/2 \pm \sqrt{3}$. The Python shortcut recovers all ten conjugates and, in particular,

$$\sqrt[5]{2} + \sqrt{3}.$$

This example shows why a small auxiliary field is often preferable to building the entire splitting field before looking for radicals.

6 Exact field representation and elementary operations

The general constructor uses an embedded number field

$$K = \mathbb{Q}(\theta) \cong \mathbb{Q}[T]/(\Phi(T)),$$

where Φ is irreducible and the particular complex value of θ is fixed. An element is stored as a rational coordinate vector

$$(a_0, \ldots, a_{D-1}), \qquad a = a_0 + a_1\theta + \cdots + a_{D-1}\theta^{D-1}, \qquad D = \deg \Phi.$$

The implementation uses SymPy's `AlgebraicField` and its algebraic-number polynomial representation. Arithmetic is exact modulo Φ; inverses use field arithmetic rather than numerical division. SymPy's number-field facilities provide primitive elements and exact membership tests [4, 5].

The embedding is as important as the quotient polynomial. An irreducible polynomial without a selected root does not identify which complex embedding is intended. Internally the primitive element may be anchored by exact radicals and isolated `CRootOf` objects. Those opaque objects are allowed inside the construction and its certificate, not in the final radical expression.

6.1 Minimal polynomials without symbolic expansion

For $a \in K$, form the coordinate vectors of $1, a, a^2, \ldots$. The first linear dependence

$$a^k = c_0 + c_1 a + \cdots + c_{k-1} a^{k-1}$$

gives

$$\operatorname{minpol}_{\mathbb{Q}}(a)(X) = X^k - \sum_{j<k} c_j X^j.$$

The first dependence occurs by $k \le D$, and minimality of k proves that this is the minimal polynomial. The implementation's `minpoly_element` performs this Krylov calculation using an incremental exact echelon basis.

The same linear-space utility later expresses a radicand in the preceding tower field. Its coordinate bookkeeping refers to the original inserted basis vectors, not just to the normalized pivot rows. This distinction matters when converting row-reduced coordinates back to symbolic monomials.

6.2 Constructing the splitting field

For the irreducible input f, enumerate its exact conjugates $\alpha_1, \ldots, \alpha_n$. Start with $\mathbb{Q}(\alpha_1)$ and adjoin each subsequent conjugate that fails an exact membership test. A final membership pass ensures that the resulting field contains them all. Since it is generated by these roots, the field is exactly their splitting field L.

Two exact construction shortcuts are included. For an irreducible cubic,

$$L = \mathbb{Q}(\alpha_1, \sqrt{\operatorname{disc}(f)}).$$

For an irreducible binomial $X^n - c$,

$$L = \mathbb{Q}(c^{1/n}, \zeta_n).$$

These shortcuts concern the representation of L. When the user forces the general method, the subsequent Galois-group, composition-series, and Kummer-generator construction still runs; the requested output is not merely taken from the binomial formula.

7 Choosing the cyclotomic base

Let $d = [L : \mathbb{Q}]$ and choose the square-free integer

$$m = \operatorname{rad}(d) = \prod_{p \mid d} p. \tag{15}$$

Set

$$E = \mathbb{Q}(\zeta_m), \qquad M = L(\zeta_m), \qquad H = \operatorname{Gal}(M/E). \tag{16}$$

Every prime dividing $|H|$ divides d, so E contains a primitive pth root of unity for every prime index that can occur in a composition series of H.

Proposition 7.1. *The group H is solvable if and only if $G = \operatorname{Gal}(L/\mathbb{Q})$ is solvable.*

Proof. Restriction identifies

$$H \cong \operatorname{Gal}(L/L \cap E).$$

The intersection $L \cap E$ is Galois over $\mathbb{Q}$. Its Galois group is an abelian quotient of the cyclotomic Galois group. Thus H is a normal subgroup of G with abelian quotient. Solvability passes to subgroups and is preserved by extensions, giving both directions. □

There is no infinite recursion in the choice of roots of unity. The base value ζ_m itself has the radical expression (1). We are not requiring that the entire calculation start with only prime-degree radical steps over $\mathbb{Q}$; we first adjoin this explicitly representable cyclotomic number and then use prime steps over E.

The implementation constructs M by exact membership or a primitive-element compositum. It verifies the dimension identity

$$[M : \mathbb{Q}] = |H|\,\varphi(m). \tag{17}$$

A failed identity is a verification failure, not an invitation to infer the intended field from numerical approximations.

8 Enumerating the Galois action

Because $M/\mathbb{Q}$ is normal, the minimal polynomial Φ of a primitive element θ splits into $D = [M : \mathbb{Q}]$ distinct linear factors over M. Every root b of Φ defines an automorphism

$$\sigma_b\left(\sum_{j=0}^{D-1} a_j\theta^j\right) = \sum_{j=0}^{D-1} a_j b^j. \tag{18}$$

This gives a direct, complete method to enumerate all automorphisms. Retain precisely those fixing the exact field element ζ_m.

Each retained automorphism permutes the original roots of f. This action is faithful: an automorphism fixing those roots and ζ_m fixes all generators of M. The implementation constructs a permutation group on the input roots and checks that the generated group has the same order as the enumerated set. Because that set is contained in its generated group, equality of cardinalities certifies closure.

8.1 A checked optimization for conjugate enumeration

Factoring a degree-D primitive polynomial over a degree-D field can dominate the calculation. The implementation first attempts to generate possible images directly from the exact expression tree of θ. Rational leaves stay fixed; isolated-root leaves are replaced by their roots in M; a rational power is handled by its defining power equation; sums and products combine candidate images.

Different occurrences can be assigned conjugates inconsistently, so this process intentionally creates an over-approximation. Every proposed image is filtered by the exact equation $\Phi(b) = 0$. The optimization is accepted only after finding *all D distinct roots*. Otherwise the algorithm falls back to factoring Φ over M. A budget of 4096 intermediate candidates and a small radical-degree guard bound this optional attempt; neither guard limits the complete fallback.

This is not numerical recognition and does not assume that an independently chosen tuple of conjugates defines a field embedding. The irreducible polynomial and the full distinct-root count supply the certificate. It is especially effective for primitive elements constructed from one root and a root of unity.

8.2 Nonsolvability detection

A derived series that stabilizes at a nontrivial subgroup proves nonsolvability. For degree at most six, the package may first use SymPy's dedicated exact Galois-group algorithm as a shortcut [4]. Its documented degree restriction belongs to that optional precheck, not to the splitting-field constructor.

If the precheck is unavailable or raises an exception, the general construction proceeds. After the full relative group H is known, Proposition 7.1 permits an exact negative answer. The supplied negative test $X^5 - X - 1$ has group order 120 and derived-group orders $120, 60$ under the small-degree routine. This identifies a genuine obstruction; it is not based on a failed search for expressions.

The negative diagnostic records the routine used and group/derived-series orders. It is not a standalone formal certificate of the Galois group. The positive radical certificates described below have a separate replay procedure that does not recompute the Galois search.

9 Deterministic Kummer generators

Assume H is solvable. Take a composition series

$$H = H_0 \triangleright H_1 \triangleright \cdots \triangleright H_s = 1, \qquad H_i/H_{i+1} \cong C_{p_i}, \tag{19}$$

where every p_i is prime and H_{i+1} is normal in H_i. The fixed fields form the ascending chain

$$E = F_0 \subset F_1 \subset \cdots \subset F_s = M, \qquad F_i = M^{H_i}, \qquad [F_{i+1} : F_i] = p_i. \tag{20}$$

This is the classical cyclic-extension route used in constructive radical algorithms; modern implementations using related Galois-theoretic constructions are discussed by Fieker and Sutherland [3].

The remaining task is algorithmic: for every step, produce a nonzero $b_i \in F_{i+1}$ such that $b_i^{p_i} \in F_i$. One must not assume that a single arbitrarily chosen resolvent will be nonzero.

9.1 The finite trace-and-Fourier search

Fix a step and abbreviate

$$N = H_{i+1}, \quad J = H_i, \quad p = [J : N], \quad F = M^J, \quad F' = M^N.$$

Choose $\sigma \in J \setminus N$. Its coset generates J/N. Let $\zeta = \zeta_p \in E \subset F$. For $j = 0, \ldots, D-1$, compute

$$t_j = \sum_{\tau \in N} \tau(\theta^j) \in F', \tag{21}$$

$$b_j = \sum_{k=0}^{p-1} \zeta^{-k} \sigma^k(t_j). \tag{22}$$

Choose the first j for which $b_j \neq 0$ in exact field arithmetic.

Lemma 9.1. *At least one of the D values b_j in (22) is nonzero.*

Proof. The powers $1, \theta, \ldots, \theta^{D-1}$ span M over $\mathbb{Q}$. The trace operator $T = \sum_{\tau \in N} \tau$ maps onto F': for $v \in F'$, $T(v/|N|) = v$. Therefore the elements t_j span F' over $\mathbb{Q}$.

On F', consider the F-linear operator

$$P = \sum_{k=0}^{p-1} \zeta^{-k} \sigma^k.$$

The p automorphisms $1, \sigma, \ldots, \sigma^{p-1}$ of F'/F are distinct. Their linear independence as field homomorphisms implies that this nontrivial linear combination is not the zero operator. If every $P(t_j)$ vanished, P would vanish on a spanning set and hence on all of F', a contradiction. □

Lemma 9.2. *For the nonzero selected element b, one has*

$$\tau(b) = b \quad (\tau \in N), \qquad \sigma(b) = \zeta b, \qquad b^p \in F.$$

Moreover, $F' = F(b)$.

Proof. Normality of N in J ensures that each $\sigma^k(t_j)$ lies in F'. Cycling the sum in (22), using $\sigma^p(t_j) = t_j$ and $\sigma(\zeta) = \zeta$, gives $\sigma(b) = \zeta b$. Hence b^p is fixed by both N and σ, which generate J. Thus $b^p \in F$.

Since $p > 1$ and $b \neq 0$, the identity $\sigma(b) = \zeta b$ shows that $b \notin F$. The extension F'/F has prime degree, so it has no proper intermediate field; therefore $F(b) = F'$. □

The release multiplies b by a nonzero rational scaling factor to clear a common coordinate content. This does not change any fixed-field or eigenvector identity. It is a normalization, not a global expression-size optimization.

9.2 Building a basis and expressing each radicand

Initially use the cyclotomic basis

$$1, \zeta_m, \ldots, \zeta_m^{\varphi(m)-1}$$

of E over $\mathbb{Q}$. Maintain two parallel lists: exact vectors in M, and formal monomials in symbols $z, r_1, \ldots, r_i$. The former are used for arithmetic; the latter will become the printed tower.

At a step with $b^p = q \in F_i$, solve an exact rational linear system to express q in the existing basis:

$$q = \sum_\nu c_\nu B_\nu, \qquad Q_i(z, r_1, \ldots, r_i) = \sum_\nu c_\nu \mathcal{B}_\nu. \tag{23}$$

Extend the basis by

$$\{B_\nu b^j : 0 \le j < p\}. \tag{24}$$

Lemma 9.2 proves independence and spanning. The implementation nevertheless checks independence as each vector is inserted. It also verifies directly that b^p belongs to the preceding span and that the eigenvector identities hold.

At the end the basis has

$$\varphi(m) \prod_i p_i = D$$

elements and spans M. Each input conjugate is expressed in this basis by exact linear algebra. Replacing the formal generators with their certified radical definitions yields the final formulas.

Theorem 9.3 (Termination of the mathematical construction)**.** *Assume the required exact polynomial factorization, algebraic-number embedding, field-membership, and finite-group primitives terminate correctly. With resource bounds removed, the general construction terminates for every irreducible $f \in \mathbb{Q}[X]$. It returns radical expressions for all roots precisely when the Galois group of f is solvable.*

Proof. There are finitely many input conjugates, so splitting-field construction terminates under the stated primitives. The finite normal compositum M has finitely many automorphisms, and their permutation group is finite. Nonsolvability is decided by a finite derived-series calculation.

In the solvable case a finite composition series exists. At each step Lemma 9.1 bounds the generator search by D trials; Lemma 9.2 makes it a valid radical step. The basis eventually spans M, so all roots have finite coordinate expressions. There are finitely many radical branches to distinguish, and exact algebraic root identification fixes them. The resulting tower consists entirely of radicals. Conversely, Theorem 3.1 excludes a radical answer in the nonsolvable case. □

This is a completeness theorem for the mathematical algorithm, conditional on its algebraic primitives. It is not a proof that a particular library version is bug-free, that a fixed time budget suffices, or that the default degree guard will admit every intermediate field.

10 Principal branches and exact conjugate selection

A power identity alone does not identify a radical. From $b^p = q$ one may conclude only

$$b = \zeta_p^k q^{1/p} \quad \text{for exactly one } k \in \{0, \dots, p-1\}, \tag{25}$$

where $q^{1/p}$ is principal and $q \neq 0$. Neglecting k is one of the easiest ways to construct a formula for the wrong conjugate.

10.1 An annihilator shared by all candidates

Inside the exact field compute $q = b^p$, then its rational minimal polynomial h_q. The polynomial

$$h(X) = \operatorname{sqf}\big(h_q(X^p)\big) \tag{26}$$

annihilates both b and every candidate $\zeta_p^k q^{1/p}$. This is proved algebraically before any approximate evaluation is used. A root-separation test can now decide which candidate agrees with b.

The implementation uses SymPy's public `Poly.same_root` routine for this step. Its implementation uses a rational root-separation bound and evaluation with controlled error [6]. That routine has an essential precondition: both arguments are already known roots of the polynomial. It is not used here as a generic test of whether a guessed expression is a root.

Numerical approximations at a modest precision are used only to order candidate comparisons. Crucially, the code evaluates the two quantities separately rather than numerically evaluating their almost-cancelling difference. Evaluating a difference of equal but syntactically dissimilar algebraic expressions can otherwise waste substantial effort trying to resolve cancellation. Every acceptance still goes through the root-separation check.

10.2 Why root separation supplies a proof

Let $\delta > 0$ be a lower bound for the distance between distinct roots of square-free h. Suppose approximations A, B to known roots a, b have individual errors less than ε, and

$$|A - B| < \varepsilon, \qquad 3\varepsilon \leq \delta.$$

Then $|a - b| < \delta$, so the known roots must be equal. Refining the approximations distinguishes equal and unequal candidates. This is fundamentally different from accepting a small floating-point residual $|f(e)|$: a residual depends strongly on conditioning and does not establish that e is algebraic of the intended value.

The trusted base still includes the algebraic and bounded-error evaluation routines. The certificate is computationally exact in that sense; it is not a formally verified theorem in Lean or another proof kernel.

10.3 Different systems have different root orderings

The Python API uses zero-based SymPy `CRootOf` ordering: real roots first, followed by its canonical complex ordering. Wolfram's `Root` indices are one-based and its complex-root ordering

must not be assumed identical. Sending a Wolfram index unchanged to Python would therefore be an incorrect interface design.

The Wolfram bridge sends only the minimal polynomial and requests all its conjugates. It decodes the returned radical expressions and uses exact `RootReduce` equality against the original Wolfram number to select one. This also avoids an arbitrary-precision decimal approximation being treated as the definition of the input embedding.

11 Straight-line towers and replayable certificates

11.1 Do not confuse a compact tower with its expanded expression

A tower is stored as

$$\begin{aligned} z &= \zeta_m, \\ r_{i+1} &= \zeta_{p_i}^{k_i} \, Q_i(z, r_1, \ldots, r_i)^{1/p_i}, \\ a_j &= A_j(z, r_1, \ldots, r_s), \end{aligned}$$

where the Q_i and A_j are rational polynomial coordinate expressions. This is a straight-line representation with shared intermediate results. The `RadicalTower` object exposes these definitions; its `expressions()` method substitutes them to produce ordinary SymPy expressions.

Expansion may duplicate a large radicand in many places. The compact tower is therefore included in the JSON response as well as the expanded radical expressions. This release does not optimize a global arithmetic circuit or minimize nesting depth. In particular, a short field-theoretic proof can coexist with unattractive expanded coefficients.

11.2 Certificate contents

A version-two positive certificate contains the rational input polynomial, the primitive polynomial of M, an exact expression anchoring the primitive embedding, the coordinate vector of ζ_m, every Kummer generator's vector, each radicand expression and branch index, and the final target vectors and coordinate expressions.

The expression protocol is a whitelist syntax, not executable code:

AST form	Interpretation
`["Q", "n", "d"]`	The rational number n/d, stored as decimal integer strings.
`["A", ...]`	Sum of recursively encoded operands.
`["M", ...]`	Product of recursively encoded operands.
`["P", base, exponent]`	A power with rational exponent.
`["S", "r1"]`	A previously defined tower symbol; permitted only where a symbol environment is supplied.

The certificate-only embedding grammar additionally permits an isolated algebraic root, encoded by rational polynomial coefficients and a zero-based `CRootOf` index. This does not evade the radical-output requirement: the public output decoder does not accept the `Root` tag. The anchor is a way to verify the desired embedding independently of the produced radical formula.

The previous temptation to identify every new primitive element by a fresh degree-D isolated-root index was avoided. That can make certificate generation more expensive than the construction itself. Preserving the exact primitive-element expression as an anchor retains the embedding without isolating an unrelated large polynomial from scratch.

11.3 Replay procedure

The verifier reconstructs $\mathbb{Q}(\theta)$ from the anchor, verifies that its actual minimal polynomial agrees with the recorded primitive polynomial, and checks irreducibility. It then performs the following substantive checks:

1. The recorded cyclotomic element satisfies its cyclotomic polynomial and has the intended principal-radical embedding.
2. Every radicand expression evaluates in the already reconstructed field elements, and the stored generator satisfies $b^p = q$ exactly.
3. The recorded branch index agrees with a fresh separation-certified branch calculation. Tower symbols cannot be redefined.
4. Every final coordinate expression reconstructs its stored target, annihilates the input polynomial, and agrees with the corresponding exact input conjugate.

This replay does not trust the Galois search log or recompute the composition series. Once a valid radical tower and its targets have been provided, the automorphisms used to discover it are unnecessary for verifying the positive result.

A linear result has a separate annihilation certificate, and an empty certificate is rejected. The live tower definitions and outputs are also bound to their certificate; standalone replay can bind a caller-supplied polynomial through `expected_polynomial`. Tests alter branches, radicands, target vectors, primitive anchors, and live tower definitions; each alteration is rejected.

12 Software organization and usage

12.1 Distribution structure

```
rootradicals/
  article/root-to-radicals.tex
  article/root-to-radicals.pdf
  python/radical_roots/core.py
  python/radical_roots/fast.py
  python/radical_roots/cli.py
  wolfram/SystematicRadicals.wl
  wolfram/SystematicRadicals.wlt
  examples/
  tests/test_radical_roots.py
  tools/run_validation.py
  validation/test-results.json
  original/FindExtension.nb
  original/FindExtension-input-cells.txt
  README.md
  pyproject.toml
  requirements.txt
```

`core.py` contains the field construction, automorphisms, prime-step resolvents, exact linear algebra, safe ASTs, and certificate replay. `fast.py` implements structural shortcuts and selected-root normalization. `cli.py` provides a process-isolated JSON interface. The Wolfram source provides native shortcuts and acts as a verified client of the Python constructor.

Only SymPy is required by the Python code. The tested dependency is pinned to version 1.14.0. The source requires Python 3.10 or later; execution in this build used Python 3.13.5.

Compatibility with every intermediate Python version is not claimed to have been tested.

12.2 Python installation and examples

From the unpacked distribution directory:

```
python -m pip install -r requirements.txt
python -m pip install -e .
python -m unittest discover -s tests -v
```

The tests and example scripts also add the local `python/` directory to the import path, so editable installation is optional for running them.

```
from sympy import symbols
from radical_roots import solve_root, solve_polynomial, build_tower, Limits

x = symbols("x")
a = solve_root(x**6 + x**4 - x**3 - x**2 - 1, 1)
print(a.expression())                    # second real root, zero based

b = solve_root(5*x**5 - 25*x**3 + 25*x + 6, 4)
print(b.expression())                    # largest real root

# Force the general construction rather than using cubic formulas.
t = build_tower(x**3 - x - 1)
assert t.verify()
print(t.metadata)
print(t.steps)                           # compact radical tower
print(t.expressions())                   # all conjugates
```

The general quintic test is explicitly forced through the same constructor:

```
t = build_tower(x**5 - 2, limits=Limits(max_field_degree=96))
assert t.verify()
assert t.metadata["splitting_degree"] == 20
assert t.metadata["chain_orders"] == [5, 1]
```

Here the relative group over the cyclotomic base has order five, even though the rational splitting field has degree twenty.

To remove the field-degree guard, use `Limits(max_field_degree=None)`. Direct Python calls use cooperative time checks: `Limits(seconds=...)` is checked between stages, not inside every library routine. For a hard wall-clock limit, use the CLI or the Wolfram bridge, both of which execute the computation in a killable worker process.

12.3 The JSON CLI

Coefficients are supplied in ascending order as exact rational strings. A minimal request is

```
{"coefficients":["6","25","0","-25","0","5"],
 "method":"auto", "seconds":300, "max_field_degree":96}
```

Run it with

```
python python/radical_roots/cli.py < examples/quintic-request.json
```

The process writes exactly one JSON response on standard output. It never evaluates source text from the request. All coefficient strings must match an integer/rational grammar. To request a particular SymPy root, add `"root_index": 4`. For an unlimited time or field-degree bound, the corresponding JSON value is `null`. The parent process enforces the requested wall-clock timeout and preserves meaningful failure statuses returned by its worker.

The CLI accepts `method` values `"auto"`, `"fast"`, and `"galois"`. The fast mode does not invoke the general constructor. Unless explicitly disabled, a newly constructed general tower is independently replayed before the response is emitted.

12.4 Wolfram Language usage

Load the package from its unpacked location:

```
Get["path/to/rootradicals/wolfram/SystematicRadicals.wl"];

a = Root[-1 - #^2 - #^3 + #^4 + #^6 &, 2];
b = Root[6 + 25 # - 25 #^3 + 5 #^5 &, 5];

SystematicToRadicals[a]
SystematicToRadicals[b]
RadicalData[a]
```

The default method tries native shortcuts first and then the Python backend. A Windows interpreter can be selected without shell quoting:

```
SystematicToRadicals[a,
  Method -> "Python",
  "PythonCommand" -> {"py", "-3"}]

SystematicToRadicals[Root[#^3 - # - 1 &, 1],
  Method -> "Galois",
  "PythonCommand" -> {"C:\\path\\to\\venv\\Scripts\\python.exe"},
  TimeConstraint -> 300]
```

The executable and its arguments are passed as a list to `RunProcess`, not interpolated into a shell command [9]. The backend script is located relative to the loaded package by default, so the directory layout should be preserved.

Wolfram method	Behavior
`Automatic`	Native shortcuts, followed by Python automatic mode.
`"Native"`	Native shortcuts only; no Python dependency, but not complete.
`"Python"`	The tested Python automatic method, with exact Wolfram selection afterward.
`"PythonFast"`	Only the tested Python structural shortcuts.
`"Galois"`	Force the complete mathematical construction in the Python backend.

Other options include `"MaxFieldDegree"`, `"PairSumMaxDegree"`, `"MaxExtensionDegree"`, `"BackendScript"`, and `"ReplayCertificate"`. Setting `"MaxFieldDegree" -> Infinity` removes that guard in the backend. A failed conversion returns a `Failure` object rather than silently returning an unchanged `Root` and making success ambiguous.

12.5 Wolfram validation status

The Wolfram source was checked statically and is accompanied by prospective `VerificationTest` cases. It was not executed in a Wolfram kernel during this build: the available evaluator connector returned HTTP 404, and no local kernel was present. This limitation is recorded in the validation metadata. The mathematical identities and the corresponding Python algorithms were executed; that is not the same as claiming a native Wolfram test run.

For a production deployment, the supplied `.wlt` suite should be run in the user's Wolfram version, especially the `RunProcess`/JSON bridge and exact selection of complex roots. The wrapper's `RootReduce` acceptance condition is deliberately independent of the backend's ordering and formula-selection logic.

13 Validation results

All 32 regression tests passed, and all six saved certificates replayed. Checks cover identities, branches, failure classification, and certificate integrity, including the original two examples, a degree-seven binomial, a degree-ten sum-of-radicals example, repeated factors, selection from a reducible polynomial containing a nonsolvable factor, a colliding pair-sum resolvent, and rejection of malformed AST input.

General-constructor tests force the Galois–Kummer path for quadratic, cyclic cubic, nonabelian cubic, biquadratic, and quintic examples. They do not merely observe that the low-degree front end knows cubic or quartic formulas. The field-construction shortcuts of Section 6 remain enabled, but automorphism enumeration, composition-series traversal, Kummer synthesis, and certificate replay all run.

Input / check	Executed path	Observed result
Original sextic $X^6 + X^4 - X^3 - X^2 - 1$	Pair-sum field	Six certified radical roots; independent minimal-polynomial check of the requested root.
Original quintic $5X^5 - 25X^3 + 25X + 6$	Dickson identity	Five certified real roots; the largest agrees with the requested fifth-root expression.
$X^7 - 2$	Dickson/binomial	All seven conjugates, without a degree-six Galois precheck.
Degree-ten polynomial of $\sqrt[5]{2} + \sqrt{3}$	Pair-sum field	All ten radical conjugates; auxiliary polynomial $S^5 - 64$.
$X^2 - 2$	General constructor	Splitting degree 2; relative chain orders $2, 1$; certificate replay.
$X^3 - 2$	General constructor	Splitting degree 6; relative chain orders $3, 1$; certificate replay.
$X^3 - 3X + 1$	General constructor	Splitting degree 3; compositum degree 6; relative chain $3, 1$.
$X^3 - X - 1$	General constructor	Splitting degree 6; compositum degree 12; nonabelian relative chain $6, 3, 1$.
$X^4 - 10X^2 + 1$	General constructor	Two successive quadratic steps; chain $4, 2, 1$; roots $\pm\sqrt{3} \pm \sqrt{2}$.
$X^5 - 2$	General constructor	Splitting/compositum degree 20; relative chain $5, 1$; full replay.
$X^5 - X - 1$	Exact Galois precheck	`NotSolvable`; group order 120, derived orders $120, 60$.

Input / check	Executed path	Observed result
Tampered certificates / hard timeout	Verifier / CLI	Altered mathematical data rejected; worker deadline reported as `ResourceLimit`.

The values in the table describe actual executed cases. They do not establish a practical upper degree bound: a favorable degree-ten input can be much easier than an unfavorable quartic compositum, and field degree and coefficient height strongly influence cost. Wall-clock timings in `validation/test-results.json` are environment-dependent observations, not performance guarantees.

Positive certificates are included for representative general cases. The tests deliberately corrupt four independent certificate components and require replay to fail. An empty certificate is also rejected. Negative and resource tests establish that the CLI does not turn a legitimate `NotSolvable` response into a generic worker error and does not report a timed-out computation as mathematically impossible.

14 Complexity, limits, and engineering tradeoffs

14.1 Why completeness is expensive

For irreducible degree n, the splitting degree satisfies $d \leq n!$, and

$$D = [M : \mathbb{Q}] \leq d\,\varphi(\mathrm{rad}(d)).$$

Even before coefficient growth is considered, a basis of this size is prohibitive for many inputs. Exact primitive-element construction, factorization over number fields, and repeated coordinate conversions can dominate the runtime. The algorithm is finite, but finiteness is a much weaker statement than practical tractability.

The pair-sum method also has a cost: its resultant has degree n^2 before diagonal removal and its resolvent has degree $n(n-1)/2$. This motivates the default input-degree bound of ten for attempting that shortcut. Raising it can help structured inputs but can also spend more effort on a shortcut than on the eventual general calculation.

The field arithmetic and linear-algebra stages are polynomial-size operations in the explicitly constructed field dimension, with bit complexity depending on coefficient heights. That observation does not make the overall algorithm polynomial in the input degree, because the field itself can be factorially large.

14.2 Resource policy

The default field-degree guard is 96. It is checked after a field extension has been constructed; it is not a promise that the primitive-element routine can predict or reject an oversized extension before consuming resources. The process-isolated CLI supplies a hard time limit around such routines. There is no portable hard memory limit in this release; operating-system or container limits are appropriate for hostile or very large inputs.

Native Wolfram shortcuts receive a bounded initial share of the automatic method's time budget. The remaining budget is passed to Python. Direct Python API calls do not automatically spawn a subprocess and therefore only provide cooperative time checks. These distinctions are explicit so that a library call that stalls inside factorization is not mistaken for an enforceable in-process deadline.

14.3 Expression size and normalization

The present implementation clears common rational coordinate content of a Kummer generator. It does not search the entire eigenline for the smallest radicand, optimize primitive elements by height, or perform general radical denesting. In the nonabelian cubic test, a correct construction can produce large expanded coefficients even though Cardano's formula is short. The default front end prevents that from being the normal user-facing path for a cubic.

The appropriate intermediate representation is the straight-line tower. Simplification should be an optional, separately verified pass: after any transformation, compare the new expression exactly with the old one. An attractive simplification that silently changes a complex branch is worse than a larger certified expression.

14.4 What the package does and does not claim

The complete path does not enumerate candidate radical strings, assume every solvable polynomial has a quadratic subfield, or stop at degree six. Its completeness comes from the fixed-field chain and the finite nonzero-resolvent lemma. Nevertheless, this release remains a research/reference implementation with ordinary software dependencies. It does not claim formal verification of SymPy, competitive performance with specialized computer-algebra systems, minimal radical depth, minimal expression size, or successful execution of every high-degree example in the supplied notebook.

The positive verifier is independent of the *search*, but not independent of the arithmetic library. It replays exact field identities and branch identification using the same SymPy trusted base. The Wolfram wrapper provides an additional independently implemented final equality check when run in an actual Wolfram kernel.

15 Further implementation work

The main performance improvement would be to avoid materializing the whole rational splitting field when only certain fixed-field generators are required. Fieker and Sutherland describe Galois-theoretic constructions for fixed fields, splitting fields, and radical solutions that provide a more specialized route [3]. Replacing the field-construction backend with such machinery need not change the public result contract or branch-verification interface.

A second improvement is height-aware choice of primitive elements and Kummer generators. The current finite search picks the first nonzero projected trace, which is easy to justify but not necessarily small. A bounded search among several nonzero candidates, scored by exact coefficient heights and tower size, would preserve correctness while improving output. A nonzero first candidate must remain available as the completeness fallback.

A pure Wolfram implementation of the full constructor is possible in principle using exact common number fields, polynomial coordinate arithmetic, permutation groups, and rational linear algebra. Care is needed with `ToNumberField`: its documented `All` form ensures the smallest common extension, whereas the default common-extension search need not do so [7]. `AlgebraicNumber` can also normalize its primitive generator to an algebraic integer [8]. A port must not assume that the stored generator remains syntactically identical to the one supplied. This release avoids claiming an untested full native port and instead supplies the executable Python construction behind a narrow checked interface.

Other extensions are useful but independent of the central theorem: more resolvents for triples or block systems, cyclotomic-recognition shortcuts, explicit number-field input domains, a streaming tower-only protocol for very large answers, and stronger standalone negative certificates. None is

silently treated as implemented here.

16 Conclusion

There is a systematic answer to the original problem. For a specified algebraic number, first use its minimal polynomial and distinguish the desired conjugate. Practical structure can often be exploited immediately: the original sextic is a quadratic lift of a cubic, and the original quintic is a Dickson polynomial. When those shortcuts fail, a solvable Galois group yields a cyclic prime chain over a suitable cyclotomic base. Exact projected traces produce a nonzero generator at each step, and exact linear algebra expresses the radicands and targets.

The difficult implementation boundary is not only deciding solvability. It is preserving embeddings, certifying radical branches, controlling intermediate fields, and reporting computational failure honestly. The accompanying package makes those boundaries explicit, provides a tested general Python path, and supplies a Wolfram Language interface whose final acceptance criterion is exact equality with the original `Root` object.

A General algorithm in executable-oriented pseudocode

```
RADICAL-TOWER(f):
  require f monic irreducible over Q
  if degree(f) = 1:
      return its rational root and a linear certificate

  optionally perform the exact small-degree Galois precheck
  L := embedded splitting field generated by all roots of f
  d := [L:Q]
  m := product of the distinct primes dividing d
  M := L(zeta_m)
  theta := an embedded primitive element of M
  D := [M:Q]

  images := all D roots in M of minpoly(theta)
  automorphisms := substitution theta -> image
  H := those automorphisms fixing zeta_m
  realize H faithfully as permutations of the roots of f
  verify |H| * phi(m) = D
  if H is not solvable:
      return NotSolvable

  H_0 > H_1 > ... > H_s = 1 := cyclic-prime composition series
  basis := [1, zeta_m, ..., zeta_m^(phi(m)-1)]
  monomials := [1, z, ..., z^(phi(m)-1)]

  for i = 0,...,s-1:
      p := [H_i:H_(i+1)]
      sigma := any element of H_i not in H_(i+1)
      for j = 0,...,D-1:
          t := sum(tau(theta^j), tau in H_(i+1))
          b := sum(zeta_p^(-k) * sigma^k(t), k=0,...,p-1)
          if b != 0: break
      verify the invariance and eigenvector identities
      q := b^p
      Q_i := exact coordinates of q in the old basis
      k_i := certify b = zeta_p^k_i * principal_root(q,p)
      append the radical definition r_(i+1)
      extend basis by old_basis * b^j, j=1,...,p-1
      extend monomials by old_monomials * r_(i+1)^j

  verify length(basis) = D
  express each input root in basis
  emit compact tower, expanded expressions, and certificate
```

B Reproducing the build and checking provenance

The tests can be rerun with `python tools/run_validation.py`, which writes a machine-readable report. The general certificate examples can be replayed independently of discovery with the supplied example script. The article builds with a standard `pdflatex` installation:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error root-to-radicals.tex
pdflatex -interaction=nonstopmode -halt-on-error root-to-radicals.tex
```

The included PDF was compiled from this source and visually inspected after rendering. No font files are distributed.

The original notebook is retained as supplied, with its provenance recorded separately. The notebook transcription is a convenience for review, not an executable conversion of every display box. The new implementation and report are distributed under the license in the archive; that license does not relicense the user's original notebook or third-party dependencies.

References

[1] V. Reshetnikov, "Why is ToRadicals not able to handle all cases? Is there a workaround?" Mathematica Stack Exchange, question 34011. https://mathematica.stackexchange.com/questions/34011/why-is-toradicals-not-able-to-handle-all-cases-is-there-a-workaround.

[2] Wolfram Research, *ToRadicals*, Wolfram Language Documentation. https://reference.wolfram.com/language/ref/ToRadicals.html.

[3] C. Fieker and N. Sutherland, *Constructions using Galois Theory*, arXiv:2010.01281, version 3, 27 October 2022. In particular, the constructions of radical solutions using cyclic extensions and roots of unity. https://arxiv.org/abs/2010.01281.

[4] SymPy Development Team, *Number Fields*, SymPy 1.14.0 Documentation. https://docs.sympy.org/latest/modules/polys/numberfields.html.

[5] SymPy Development Team, *Polynomial Domains Reference*, especially `AlgebraicField`, SymPy 1.14.0 Documentation. https://docs.sympy.org/latest/modules/polys/domainsref.html.

[6] SymPy Development Team, *Polynomials Manipulation Module Reference*, especially `Poly.same_root`, and its linked implementation source, SymPy 1.14.0 Documentation. https://docs.sympy.org/latest/modules/polys/reference.html.

[7] Wolfram Research, *ToNumberField*, Wolfram Language Documentation. https://reference.wolfram.com/language/ref/ToNumberField.html.

[8] Wolfram Research, *AlgebraicNumber*, Wolfram Language Documentation. https://reference.wolfram.com/language/ref/AlgebraicNumber.html.

[9] Wolfram Research, *RunProcess*, Wolfram Language Documentation. https://reference.wolfram.com/language/ref/RunProcess.html.

Documentation and online sources were consulted on 8 September 2026. The notebook supplied with the request is the primary source for the code-review observations in Section 2.